

Natural Language Processing

From Text to Intelligence

Module: Text Processing Fundamentals

A Comprehensive Guide to Mathematical Abstractions and Algorithms

Prepared for Student Learning

January 23, 2026

Contents

1	The Art of Text Preprocessing	2
1.1	Introduction	2
1.2	The Pipeline of Purification	2
2	Representing Text as Numbers	4
2.1	The Bag-of-Words (BoW) Assumption	4
2.2	TF-IDF: The Mathematics of Rarity	4
2.3	The TF-IDF Abstraction Matrix	4
3	Meaning in Context: Semantic Correlation	6
3.1	The Semantic Arrow	6
4	Probabilistic Language Models	7
4.1	Markov Models and Transition Matrices	7
4.1.1	The Transition Matrix	7
4.1.2	State Transition Graph	7
4.2	Simulation: The Chatbot's Prediction	8
5	The Neural Revolution: Word2Vec	9
5.1	Mathematical Abstraction of Word2Vec	9
5.2	Neural Architecture Diagram	9
5.3	Vector Space and Cosine Similarity	10
5.4	Conclusion	10

The Art of Text Preprocessing

Introduction

Language is the primary vessel of human intelligence, yet to a computer, language is nothing more than an unstructured stream of bytes. Before we can teach a machine to understand irony, sentiment, or translation, we must first bridge the gap between human expression and mathematical representation. This chapter begins our journey at the foundation: Text Preprocessing. This is not merely a "cleanup" phase; it is the process of distilling noise into signal.

The Pipeline of Purification

Imagine a raw sentence captured from the wild: "Pak Andi membeli 2 kilo madu di pasar!". It contains capitalization, punctuation, and grammatical affixes that often obfuscate the core semantic meaning. Our goal is to normalize this into a standard form that a model can digest.

We begin with **Normalization**. The diversity of character representations (such as capitalization) introduces unnecessary complexity. By converting all text to lowercase, we ensure that "Madu" and "madu" are treated as the exact same entity. Furthermore, we employ Regular Expressions (Regex) to surgically remove non-alphabetic characters. Digits are often replaced with a placeholder or removed entirely if quantity is irrelevant to the specific task, while punctuation marks which carry syntactic but often not semantic weight in simple models are stripped away.

Once the text is normalized, we proceed to **Tokenization**. This is the act of atomic segmentation. A sentence is not processed as a whole; it is broken down into its constituent atoms, called tokens. In many Western languages, whitespace serves as a natural delimiter. Thus, our sentence transforms from a single string into a list of discrete units: ['pak', 'andi', 'membeli', 'kilo', 'madu', 'pasar'].

However, not all tokens are created equal. Some words serve merely as syntactic glue, binding nouns and verbs together without adding significant meaning. Words like "yang", "dan", "di", or "ke" are known as **Stopwords**. In the context of keyword extraction or topic modeling, these words introduce high-frequency noise. By filtering them out using a predefined set, we reduce the dimensionality of our data and allow the model to focus on the content-bearing words.

Finally, we arrive at the most linguistically complex step: **Stemming**. In languages like Indonesian, words are rich with morphology. The word "membeli" (buying) and "beli" (buy) share the same root semantics. If we treat them as different words, our model fails to learn their connection. Stemming algorithms crudely but effectively chop off affixes to reveal the root form. For instance, suffixes like *-lah*, *-kah*, *-pun*, *-nya*, *-kan*, *-i*, and *-an* are systematically removed. "Membeli" becomes "bel", "makanan" becomes "makan". This conflation reduces the vocabulary size significantly, making the learning process more efficient.

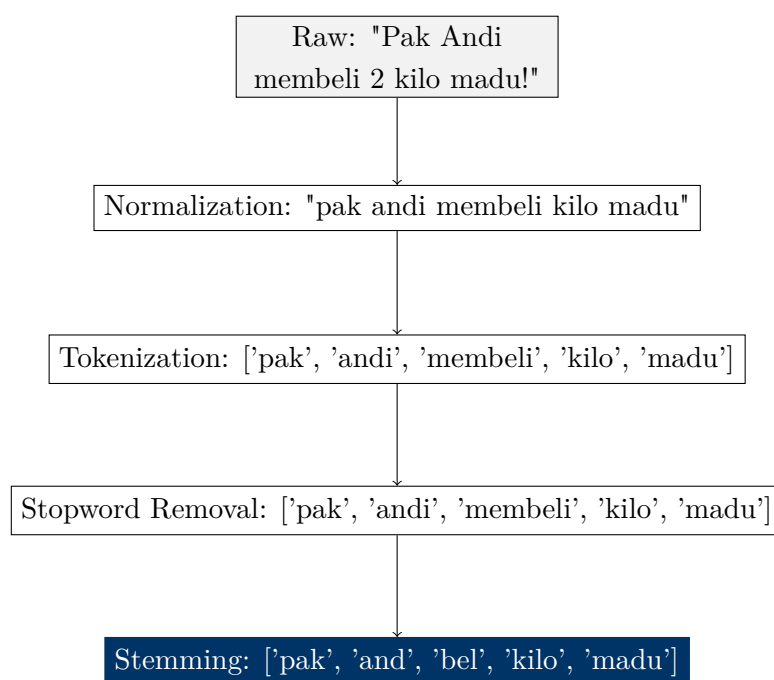


Figure 1.1: The Pipeline of Preprocessing transformations.

Representing Text as Numbers

The Bag-of-Words (BoW) Assumption

Now that we have clean tokens, we face a fundamental problem: mathematical models operate on vectors, not strings. The simplest approach to vectorization is the Bag-of-Words model. As the name suggests, we disregard grammar and word order entirely, throwing all words into a "bag" and counting their frequencies. This approach is intuitive but suffers from a critical flaw: it treats common words as important. If the word "madu" appears 50 times, it dominates the vector, potentially overshadowing rarer, more specific terms.

TF-IDF: The Mathematics of Rarity

To counter the bias of frequency, we introduce the **Term Frequency-Inverse Document Frequency (TF-IDF)** weighting scheme. This algorithm is built on a profound insight: *a word is important if it appears frequently in a document, but rarely across the entire corpus.*

Let us break down the formula step-by-step. First, **Term Frequency (TF)** measures local importance. For a term t in a document d :

$$TF(t, d) = \frac{\text{count}(t, d)}{\text{total words in } d}$$

Second, **Inverse Document Frequency (IDF)** measures global rarity. If we have N total documents and the term t appears in $df(t)$ of them:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

The logarithm dampens the effect of frequency scaling, and the inverse ratio ensures that ubiquitous words (like "the" or "dan") approach a weight of zero. The final weight is:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

The TF-IDF Abstraction Matrix

Let us visualize the TF-IDF measurement for our "Pak Andi" corpus.

- Doc 1: "Pak Andi membeli 2 kilo madu di pasar."
- Doc 2: "Madu alami sangat baik untuk kesehatan!"
- Doc 3: "Apakah Andi suka minum teh dengan madu?"
- Doc 4: "Teh hangat dan madu bisa membantu meredakan batuk."

Assume $N = 4$. Let's focus on three tokens: **madu**, **batuk**, and **andi**.

- **madu**: Appears in 4 docs ($df = 4$). $IDF = \log(4/4) = 0$.
- **batuk**: Appears in 1 doc ($df = 1$). $IDF = \log(4/1) \approx 0.602$.
- **andi**: Appears in 2 docs ($df = 2$). $IDF = \log(4/2) \approx 0.301$.

The resulting Term-Document Matrix illustrates how the "weight" of meaning is distributed. Notice how "madu" becomes weightless despite being frequent, while "batuk" gains high significance in its specific context.

Document	Token: madu	Token: batuk	Token: andi
Doc 1	$TF \times 0 = \mathbf{0.00}$	$0 \times 0.602 = \mathbf{0.00}$	$TF \times 0.301 \approx \mathbf{0.05}$
Doc 2	$TF \times 0 = \mathbf{0.00}$	$0 \times 0.602 = \mathbf{0.00}$	$0 \times 0.301 = \mathbf{0.00}$
Doc 3	$TF \times 0 = \mathbf{0.00}$	$0 \times 0.602 = \mathbf{0.00}$	$TF \times 0.301 \approx \mathbf{0.05}$
Doc 4	$TF \times 0 = \mathbf{0.00}$	$TF \times 0.602 \approx \mathbf{0.10}$	$0 \times 0.301 = \mathbf{0.00}$

Table 2.1: Abstracted TF-IDF Matrix showing the suppression of common words.

Meaning in Context: Semantic Correlation

The Semantic Arrow

Language is not a bag of isolated atoms; it is a web of connections. Words obtain meaning through their neighbors. This is the ****Distributional Hypothesis****. When we process text, we can visualize semantic correlation as a directed graph where tokens "point" to the words they frequently modify or accompany.

Consider the ambiguity of the word "Bank". In our corpus, "Madu" (honey) is the central node. It correlates with "Alami" (natural) and "Pasar" (market), but distinctly separates into different contexts.

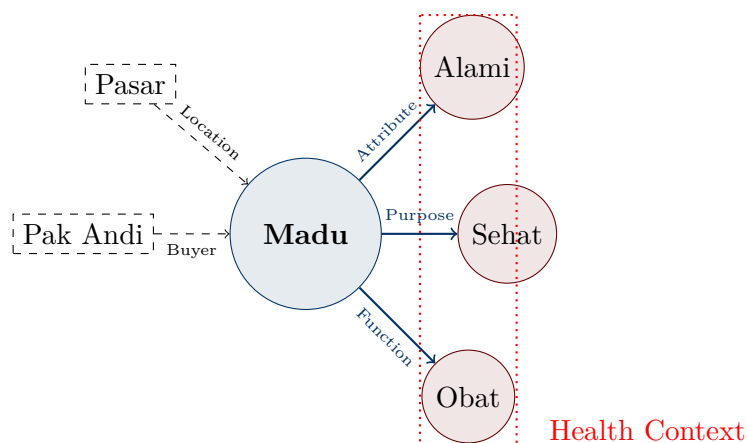


Figure 3.1: Semantic Correlation Diagram: Tokens act as pointers to context.

By mapping these arrows, we begin to see that "Madu" is not just a string, but a concept defined by "Health" (Alami, Sehat) and "Commerce" (Pasar, Pak Andi).

Probabilistic Language Models

Markov Models and Transition Matrices

Text is a sequence. The probability of the word "batuk" appearing depends heavily on the previous word being "meredakan". This sequential dependency is modeled using **Markov Chains**. In a Bigram model ($N=2$), we assume the probability of a word depends *only* on the immediate predecessor.

The Transition Matrix

Let us build a real Probability Transition Matrix from our dataset. We focus on the token "**Madu**" and what follows it.

- Doc 1: "... madu **di** pasar"
- Doc 2: "Madu **alami** ..."
- Doc 3: "... dengan madu" (End of Sentence / STOP)
- Doc 4: "... dan madu **bisa** ..."

We can abstract this into a matrix P where $P_{ij} = P(\text{word}_j | \text{word}_i)$.

Current / Next	di	alami	bisa	EOS	pasar
Madu	0.25	0.25	0.25	0.25	0.00
Teh	0.00	0.00	0.00	0.00	0.00
Di	0.00	0.00	0.00	0.00	1.00

Table 4.1: Transition Probability Matrix (Simplified).

State Transition Graph

We can visualize the "brain" of a Markov Chatbot as a directed graph where edges represent probabilities.

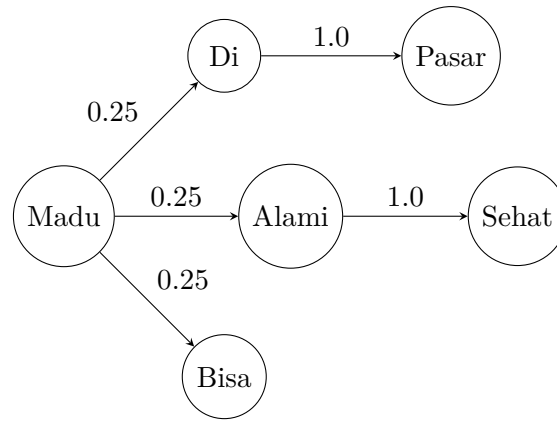


Figure 4.1: Markov State Transition Graph for Next-Token Prediction.

Simulation: The Chatbot's Prediction

How does a chatbot predict the next token? It performs a **Weighted Random Sampling**.

1. **Input:** User types "Madu".
2. **Lookup:** The model looks at the row for "Madu" in the transition matrix.
3. **Distribution:** It sees: {di : 25%, alami : 25%, bisa : 25%, EOS : 25%}.
4. **Sampling:** It generates a random number between 0 and 1.
 - If $0.0 \leq r < 0.25$, it picks "di".
 - If $0.25 \leq r < 0.50$, it picks "alami".

Study Case Measurement: If the model predicts "alami" but the actual next word in the test set is "di", we calculate the **Perplexity** or **Cross-Entropy Loss**. Since the model assigned a probability of 0.25 to the correct word "di", the loss is $-\log(0.25)$. A perfect model would have assigned 1.0, yielding a loss of 0.

The Neural Revolution: Word2Vec

Mathematical Abstraction of Word2Vec

Word2Vec is not a counter; it is a classifier. Specifically, the **Skip-gram** model tries to predict context words (w_c) given a target word (w_t).

We define two vectors for every word w :

1. v_w : The vector when w is the center word.
2. u_w : The vector when w is a context word.

The probability of seeing a context word c given center word t is defined by the **Softmax** function:

$$P(c|t) = \frac{\exp(u_c^T v_t)}{\sum_{k=1}^V \exp(u_k^T v_t)}$$

Our objective function $J(\theta)$ is to maximize this probability across the entire corpus.

$$J = - \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j}|w_t)$$

Neural Architecture Diagram

The "neural" aspect is a simple shallow network. It has an Input Layer (One-hot encoding), a Projection (Hidden) Layer (the embedding lookup), and an Output Layer (probabilities).

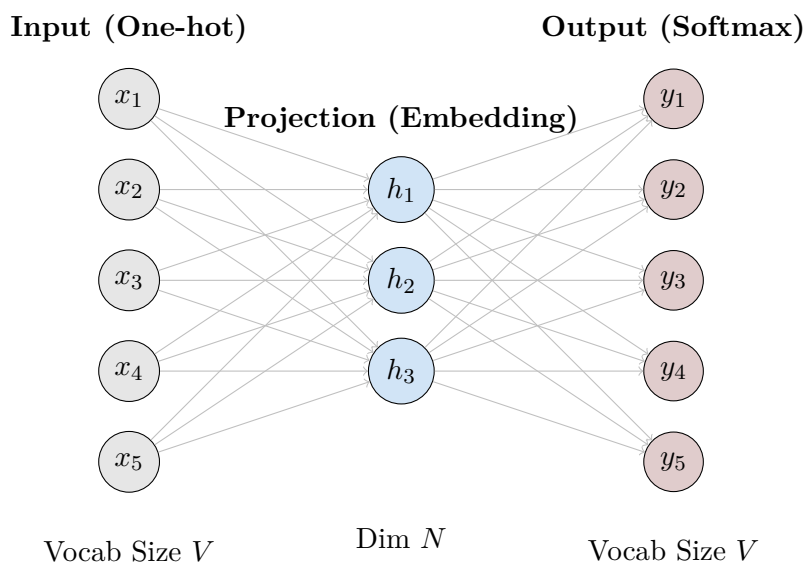


Figure 5.1: Word2Vec (Skip-gram) Architecture. The weights between Input and Projection are the embeddings.

Vector Space and Cosine Similarity

Once trained, words are mapped to a vector space. We use **Cosine Similarity** to measure their relationship. Geometrically, this is the cosine of the angle θ between two vectors.

$$\text{similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

A famous property of these spaces is the ability to capture semantic analogies through vector arithmetic. We can intuitively express the relationship between gender and royalty mathematically.

If we assume the "distance" or "difference" between King and Queen represents the concept of gender, it should be similar to the difference between Man and Woman:

$$\vec{v}_{King} - \vec{v}_{Queen} \approx \vec{v}_{Man} - \vec{v}_{Woman}$$

By rearranging this equation, we can derive the vector for "Queen" using simple algebra:

$$\vec{v}_{Queen} \approx \vec{v}_{King} - \vec{v}_{Man} + \vec{v}_{Woman}$$

Below is a visualization of a 2D vector space. Notice how "King" and "Queen" (both royalty) share a similar direction (small angle θ), while "King" and "Woman" might have a distinct angle representing their gender difference, yet the spatial relationship preserves the analogy structure.

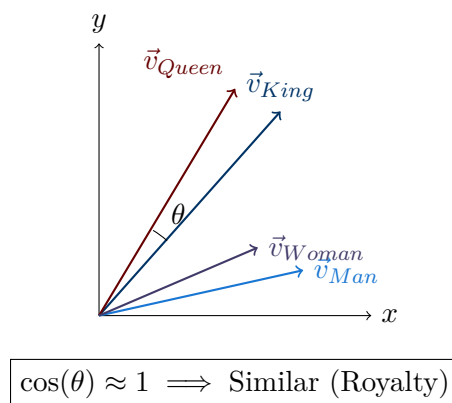


Figure 5.2: Vector Space Visualization demonstrating the King-Queen relationship.

Conclusion

We have moved from counting words in a bag to understanding them as vectors in a high-dimensional space. Whether through the probabilistic paths of Markov chains or the neural weights of Word2Vec, our goal remains the same: to mathematically encode the essence of human language.